

My responses were based on:

- General knowledge of RStudio Server, WSL2, and R package management
- Standard Linux/Debian system administration practices
- Official RStudio and R documentation patterns

1. Update Your System

bash

```
sudo apt update && sudo apt upgrade -y
sudo apt install wget gdebi-core dirmngr -y
```

2. Install R

Add the CRAN repository and install R:

bash

Add CRAN GPG key

```
sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys 51716619E084DAB9
```

Add CRAN repository for Debian

```
echo "deb https://cloud.r-project.org/bin/linux/debian $(lsb_release -cs)-cran40/" | sudo tee -a
/etc/apt/sources.list
```

Update and install R

```
sudo apt update
```

```
sudo apt install r-base r-base-dev -y
```

Verify installation

```
R --version
```

3. Install RStudio Server

Download and install the latest RStudio Server for Debian:

bash

Download (adjust URL for latest version from <https://posit.co/download/rstudio-server/>)

```
wget https://download2.rstudio.org/server/debian9/x86_64/rstudio-server-latest-amd64.deb
```

Install

```
sudo gdebi rstudio-server-latest-amd64.deb
```

4. Start RStudio Server

Since WSL2 may not have systemd enabled by default:

bash

Try standard start

```
sudo rstudio-server start
```

If that fails, use direct binary:

```
sudo /usr/lib/rstudio-server/bin/rsrserver
```

5. Access RStudio in Your Browser

Open your Windows browser and navigate to:

Code

```
http://localhost:8787
```

Login with your **WSL2 Debian username and password**.

Optional: Enable Persistent Service

To make RStudio Server start automatically, enable systemd in WSL2:

Edit C:\Users\<YourUser>\.wslconfig:

Code

```
[interop]
```

```
appendWindowsPath = true
```

```
[boot]
```

```
systemd = true
```

Then restart WSL:

PowerShell

```
wsl --shutdown
```

Then you can use standard systemd commands:

```
bash
```

```
sudo systemctl start rstudio-server
```

```
sudo systemctl enable rstudio-server
```

Any specific issues or additional packages you need to install in RStudio?

You said: how would i add packages to installed

how would i add packages to installed R

Copilot said: Here are the main ways to add

Here are the main ways to add packages to your installed R:

1. Install from CRAN (Most Common)

Via R Console/RStudio

Open R or RStudio and run:

```
R
```

```
install.packages("package_name")
```

For multiple packages:

```
R
```

```
install.packages(c("ggplot2", "dplyr", "tidyr"))
```

From Command Line

```
bash
```

```
Rscript -e 'install.packages("package_name")'
```

2. Install from Bioconductor (for bioinformatics packages)

```
R
```

```
if (!require("BiocManager", quietly = TRUE))
```

```
  install.packages("BiocManager")
```

```
BiocManager::install("package_name")
```

3. Install from GitHub

First install devtools:

```
R
```

```
install.packages("devtools")
```

```
devtools::install_github("username/repo")
```

Or use remotes (lighter alternative):

```
R
```

```
install.packages("remotes")
remotes::install_github("username/repo")
```

4. Install from Local File

```
R
install.packages("/path/to/package.tar.gz", repos = NULL, type = "source")
```

5. Install System Dependencies (if needed)

Some R packages require system libraries. For Debian/WSL2:

```
bash
```

```
# Common dependencies for R packages
```

```
sudo apt install libcurl4-openssl-dev libssl-dev libxml2-dev -y
```

```
# For spatial packages
```

```
sudo apt install libgdal-dev libgeos-dev libproj-dev -y
```

```
# For image processing
```

```
sudo apt install libmagick++-dev -y
```

6. Check Installed Packages

```
R
```

```
# List all installed packages
```

```
library()
```

```
# Check if a package is installed
```

```
"package_name" %in% rownames(installed.packages())
```

```
# Get package version
```

```
packageVersion("package_name")
```

Common R Packages to Get Started

```
R
```

```
install.packages(c(
  "tidyverse", # Data wrangling and visualization
  "ggplot2",   # Advanced graphics
  "shiny",     # Interactive web apps
  "rmarkdown", # Dynamic documents
  "devtools",  # Package development
  "rstudioapi" # RStudio integration
))
```

Pro Tip: If you're installing many packages, create a script file and run it:

```
bash
```

```
Rscript install_packages.R
```

Where install_packages.R contains:

```
R
```

```
packages <- c("ggplot2", "dplyr", "tidyr", "shiny")
```

```
install.packages(packages)
```

What packages are you looking to install?

You said: how do i set username and password

how do i set username and password for rstudio server

Copilot said: Here are the ways to set username

Here are the ways to set username and password for RStudio Server in WSL2:

1. Create a New WSL2 User (Recommended)

RStudio Server uses your **WSL2 system users** for authentication. Create a dedicated user:

```
bash
```

```
# Create new user
```

```
sudo useradd -m -s /bin/bash rstudio_user
```

```
# Set password for the user
```

```
sudo passwd rstudio_user
```

Then login to RStudio at <http://localhost:8787> with:

- **Username:** rstudio_user
- **Password:** (the password you just set)

2. Use Your Existing WSL2 User

If you already have a WSL2 user, just use that:

```
bash
```

```
# Set/reset password for current user
```

```
sudo passwd $USER
```

Then login with your existing WSL2 credentials.

3. Change RStudio Server Port (Optional)

If you want to customize the server, edit the config file:

```
bash
```

```
sudo nano /etc/rstudio/rserver.conf
```

Add or modify these lines:

```
Code
```

```
# Change port (default is 8787)
```

```
www-port=8787
```

```
# Bind to specific address
```

```
www-address=127.0.0.1
```

```
# Authentication timeout (in minutes)
```

```
session-timeout-minutes=30
```

Save and restart:

```
bash
```

```
sudo rstudio-server restart
```

Or if systemd isn't available:

```
bash
```

```
sudo pkill rserver
```

```
sudo /usr/lib/rstudio-server/bin/rserver
```

4. Reset a User Password

If you forget the password:

```
bash
```

```
sudo passwd username
```

5. View RStudio Server Logs

If login fails, check the logs:

```
bash
```

```
sudo tail -f /var/log/rstudio-server/secure-auth.log
```

Quick Setup Example:

```
bash
```

```
# Create user
```

```
sudo useradd -m -s /bin/bash myuser
```

```
# Set password
```

```
sudo passwd myuser
```

```
# Start RStudio Server
```

```
sudo /usr/lib/rstudio-server/bin/rsserver
```

```
# Access at http://localhost:8787
```

```
# Login: myuser / (your password)
```

That should cover it! Let me know if you have issues logging in.

You said: how do i mount shared folder between

how do i mount shared folder between host and rstudio-server

Copilot said: Here's how to mount a shared folder

Here's how to mount a shared folder between your Windows host and RStudio Server in WSL2:

1. Access Windows Folders from WSL2 (Easiest)

Windows drives are automatically mounted in WSL2 under /mnt/:

```
bash
```

```
# View your Windows C: drive
```

```
ls /mnt/c
```

```
# View your Documents folder
```

```
ls /mnt/c/Users/YourUsername/Documents
```

You can access these directly from RStudio without additional setup.

2. Create a Symbolic Link (Recommended)

Link your Windows folder to your WSL2 home for easier access:

```
bash
```

```
# Create symlink to Windows Documents
```

```
ln -s /mnt/c/Users/YourUsername/Documents ~/windows_docs
```

```
# Create symlink to any Windows folder
```

```
ln -s /mnt/c/Users/YourUsername/MyProjects ~/my_projects
```

Now access from RStudio:

```
R
```

```
# In R/RStudio
```

```
list.files("~/windows_docs")
```

```
list.files("~/my_projects")
```

3. Set Up a Dedicated Shared Folder

Step 1: Create folder on Windows

Code

```
C:\Users\YourUsername\RStudio_Projects
```

Step 2: Create symlink in WSL2

```
bash
```

```
ln -s /mnt/c/Users/YourUsername/RStudio_Projects ~/shared
```

Step 3: Use in RStudio

In RStudio, set your working directory:

```
R
```

```
setwd("~/shared")
```

```
# Or use absolute path
```

```
setwd("/mnt/c/Users/YourUsername/RStudio_Projects")
```

```
# List files
```

```
list.files()
```

4. Mount with Better Permissions (Advanced)

If you have permission issues, remount with different options:

```
bash
```

```
# Check current mount options
```

```
mount | grep /mnt/c
```

```
# Remount with metadata support (preserves Unix permissions)
```

```
sudo mount -t drvfs C: /mnt/c -o metadata
```

Add to /etc/wsl.conf to make it permanent:

```
bash
```

```
sudo nano /etc/wsl.conf
```

Add:

```
INI
```

```
[interop]
```

```
appendWindowsPath = true
```

```
[automount]
```

```
enabled = true
```

```
root = /mnt
```

```
options = "metadata,umask=22"
```

Restart WSL:

```
PowerShell
```

```
wsl --shutdown
```

5. Quick RStudio Workflow

```
R
```

```
# Set working directory to shared folder
```

```
setwd("~/shared")
```

```
# Create and save files
```

```
my_data <- data.frame(x = 1:10, y = rnorm(10))  
write.csv(my_data, "my_analysis.csv")
```

```
# Files are immediately visible on Windows!
```

Troubleshooting

Permission Denied Errors

```
bash
```

```
# Check permissions
```

```
ls -la /mnt/c/Users/YourUsername/RStudio_Projects
```

```
# Change ownership if needed
```

```
sudo chown -R $USER:$USER ~/shared
```

Slow File Access

This is normal with /mnt/c. For better performance, keep your project files **inside WSL2** and sync to Windows periodically.

File Compatibility

Avoid editing Windows files directly from WSL if they're in use on Windows (can cause corruption). Copy them first or edit inside WSL.

Best Practice Setup:

```
bash
```

```
# Create projects directory in WSL (faster)
```

```
mkdir -p ~/projects
```

```
# Link to Windows for backup/access
```

```
ln -s ~/projects /mnt/c/Users/YourUsername/WSL_Projects
```

```
# Work in ~/projects from RStudio
```

```
# Files automatically appear in Windows too
```